

1. Bối cảnh bài toán

Hệ thống gồm một tập client IDS phân tán:

[
 $\mathcal{C} = \{C_1, C_2, \dots, C_K\}$
]

Mỗi client có thể là một mạng doanh nghiệp, gateway, sensor, SOC node hoặc edge IDS. Không tồn tại server trung tâm. Các client trao đổi mô hình, metadata và thống kê học được với các neighbor trong đồ thị truyền thông phi tập trung:

[
 $G_t = (V, E_t)$
]

Tại thời điểm/task (t), client (C_i) tự thu thập và gán nhãn tập dữ liệu:

[
 $D_i^t = \{(x, y)\}$
]

Trong đó (x) là flow/log/packet/telemetry feature và (y) là nhãn đã biết, ví dụ benign, DoS, Botnet, PortScan, Web Attack, Exfiltration. Điểm cần nhấn mạnh trong bài là **dữ liệu task mới không được tạo tập trung**, không được giả định có sẵn toàn cục, mà do từng client tự thu thập và tự gán nhãn theo môi trường vận hành của mình.

Một ranh giới quan trọng: phương pháp này **không giải quyết unknown-class discovery**. Tức là hệ thống không cần phát hiện tự động “đây là lớp chưa biết”. Trước khi bước học liên tục bắt đầu, client đã có dữ liệu task mới với nhãn tương ứng. Nếu trong thực tế có traffic lạ, việc phân tích, điều tra và gán nhãn nó thành một lớp mới thuộc về pipeline vận hành/SOC, không phải đóng góp chính của phương pháp đề xuất.

2. Ý tưởng tổng quát

NICE-DFL-IDS kết hợp ba lớp cơ chế:

Thứ nhất, continual local learning theo NICE.

Mỗi client dùng một kiến trúc IDS có neuron được chia theo tuổi: neuron dự trữ, neuron trẻ đang học, và neuron trưởng thành đã bị đóng băng để giữ tri thức cũ. Khi task mới đến, client chỉ cập nhật phần neuron trẻ, chọn neuron quan trọng dựa trên kích hoạt, sau đó làm trưởng thành chúng. NICE gốc chọn tập neuron nhỏ nhất nhưng vẫn giữ được phần lớn tổng kích hoạt

của layer, thường theo ngưỡng (τ), và dùng freezing/pruning để tránh neuron cũ bị can thiệp bởi task mới.

Thứ hai, **context-aware decentralized clustering**.

Thay vì cluster client chỉ dựa trên gradient hoặc độ gần mạng, mỗi client tạo một **NICE Context Capsule** gồm: mẫu kích hoạt neuron, tuổi neuron, neuron importance, class/task prototypes, độ tin cậy local validation, và mức sử dụng capacity. Các capsule này cho biết client đang học loại ngữ cảnh tấn công nào và tri thức nào đã ổn định. Client có context giống nhau sẽ được gom vào cùng cụm cộng tác.

Thứ ba, **age-aware decentralized aggregation**.

Aggregation không trung bình toàn bộ mô hình một cách mù quáng như FedAvg. Chỉ những phần mô hình tương thích về context, nhãn, neuron age và activation signature mới được aggregate. Neuron trưởng thành đại diện cho tri thức cũ được bảo vệ; neuron trẻ học task mới được chia sẻ có điều kiện trong cụm phù hợp.

3. Kiến trúc local IDS tại mỗi client

Mỗi client duy trì một mô hình:

```
[  
fi(x)=hi(gi(x))  
]
```

Trong đó (g_i) là encoder trích xuất biểu diễn từ flow/log, còn (h_i) là classifier. Encoder có thể là MLP, 1D-CNN, Transformer nhẹ hoặc GNN tùy loại dữ liệu IDS. Với dữ liệu flow/tabular, MLP hoặc TabTransformer là lựa chọn thực dụng hơn.

Mỗi neuron/channel trong layer (l) có tuổi:

```
[  
a{i,l,u} ∈ {0,1,2,...}  
]
```

Trong đó:

- ($a=0$): neuron dự trữ, chưa được dùng ổn định.
- ($a=1$): neuron trẻ, đang học task hiện tại.
- ($a \geq 2$): neuron trưởng thành, đóng vai trò bộ nhớ cho task cũ.

Output neuron của một lớp tấn công được gán tuổi khi lớp đó lần đầu xuất hiện trong dữ liệu đã gán nhãn của client. Ví dụ, nếu client (C_i) lần đầu học lớp “Botnet” ở task (t), output neuron

tương ứng với “Botnet” được xem là neuron trẻ trong task đó, sau đó trưởng thành khi task kết thúc.

4. Local continual learning dựa trên NICE

Khi client (C_i) nhận task mới (D_i^t), quá trình học gồm các bước sau.

4.1. Kích hoạt neuron dự trữ

Tất cả neuron ($a=0$) tạm thời được chuyển sang ($a=1$) để cung cấp capacity học task mới. Tuy nhiên, không phải tất cả neuron trẻ sẽ được giữ lại.

4.2. Huấn luyện có kiểm soát can thiệp

Client huấn luyện trên dữ liệu task mới:

$$\begin{aligned} & [\\ & \min_{\{\theta_i\}} \mathcal{L}_{CE}(f_i(x), y), \quad (x, y) \in D_i^t \\ &] \end{aligned}$$

Nhưng chỉ cập nhật những kết nối thuộc vùng plastic. Các neuron trưởng thành ($a \geq 2$) được đóng băng. Các kết nối từ neuron trẻ sang neuron già hơn bị prune để tránh việc neuron cũ phụ thuộc vào neuron mới, tương tự cơ chế tránh interference của NICE.

Điểm cần viết rõ trong paper: loss chỉ áp dụng trên các output class có mặt trong batch/task hiện tại, hoặc trên tập output hợp lệ của context tương ứng. Điều này giúp tránh việc các output neuron cũ bị phạt sai khi client đang học lớp mới.

4.3. Chọn neuron quan trọng theo activation

Sau mỗi (p) epoch, client tính activation của neuron trẻ trên một tập con dữ liệu task mới:

$$\begin{aligned} & [\\ & I_{\{i,l,u\}}^t = \sum_{x \in \tilde{D}^t} A(n\{l,u\}, x) \\ &] \end{aligned}$$

Với mỗi layer (l), chọn tập neuron trẻ nhỏ nhất ($S_{\{i,l\}}^t$) sao cho:

$$\begin{aligned} & [\\ & \sum_{u \in S_{\{i,l\}}^t} I_{\{i,l,u\}}^t \\ & \geq \\ & \tau \end{aligned}$$

$$\sum_{u: a_{\{i,l,u\}}=1} I_{\{i,l,u\}}^t$$

Các neuron trẻ không được chọn sẽ quay lại trạng thái dự trữ ($a=0$). Các neuron được chọn tiếp tục học task hiện tại và sau task sẽ trưởng thành. NICE gốc dùng activation như chỉ báo importance và chọn greedy theo activation giảm dần .

4.4. Cập nhật context memory

Client tạo vector kích hoạt nhị phân:

$$b_i(x) = \mathbb{1}[A_i(x) > \mu_l + \sigma_l]$$

Trong đó ($A_i(x)$) là activation toàn mạng hoặc theo layer, còn (μ_l, σ_l) là thống kê ngưỡng của layer. NICE gốc dùng threshold theo mean + standard deviation để tạo binary activation vector và dùng các vector này cho context-detector .

Với mỗi class hoặc task, client lưu prototype:

$$P_{\{i,c\}}^t = \frac{1}{|D_{\{i,c\}}^t|} \sum_{x \in D_{\{i,c\}}^t} b_i(x)$$

Đây không phải dữ liệu thô, mà là chữ ký ngữ cảnh học được từ activation của mô hình.

5. NICE Context Capsule

Sau local training, client (C_i) không chỉ gửi model update. Nó gửi một gói metadata gọi là **NICE Context Capsule**:

$$\Psi_i^t = \{ P_{\{i,c\}}^t, M_i^t, A_i^t, R_i^t, Q_i^t, H_i^t, n_i^t$$

```
}
]
```

Trong đó:

- $(P_{i,c}^t)$: activation prototype theo class/task.
- (M_i^t) : age mask, cho biết neuron nào là trẻ, neuron nào trưởng thành.
- (A_i^t) : neuron importance score dựa trên activation.
- (R_i^t) : local reliability, ví dụ validation accuracy, loss, F1, calibration score.
- (Q_i^t) : context-detector parameters hoặc context probability summary.
- (H_i^t) : capacity histogram, tỷ lệ neuron đã trưởng thành theo layer.
- (n_i^t) : số mẫu local theo class/task.

Đây là phần mở rộng quan trọng so với NICE gốc. Trong NICE-DFL-IDS, thông tin activation/age/context không chỉ dùng để suy luận cục bộ, mà còn trở thành tín hiệu điều phối cộng tác trong decentralized FL.

6. Decentralized clustering dựa trên NICE information

Mỗi client nhận capsule từ neighbor và tính độ tương đồng:

[

s_{ij}

$\lambda_1 \cos(P_i, P_j)$

+

$\lambda_2 J(M_i, M_j)$

+

$\lambda_3 O(Y_i, Y_j)$

+

$\lambda_4 C(H_i, H_j)$

+

$\lambda_5 R_j$

$\lambda_6 D(\Delta_i, \Delta_j)$
]

Trong đó:

- $(\cos(P_i, P_j))$: độ giống nhau giữa activation prototypes.
- $(J(M_i, M_j))$: Jaccard similarity giữa age masks hoặc selected-neuron masks.
- $(O(Y_i, Y_j))$: mức overlap giữa các nhãn đã học.
- $(C(H_i, H_j))$: độ tương thích capacity giữa hai client.
- (R_j) : độ tin cậy local của neighbor.
- $(D(\Delta_i, \Delta_j))$: độ lệch update hoặc anomaly score.

Client xây dựng đồ thị cộng tác cục bộ:

[
 $E_{ij}=1 \quad \text{nếu} \quad s_{ij} > \delta$
]

Sau đó thực hiện clustering phi tập trung bằng label propagation, gossip-based community detection hoặc Louvain cục bộ. Kết quả là mỗi client thuộc về một cụm cộng tác động:

[
 $\mathcal{G}_i = \{C_j \mid C_j \text{ có context tương thích với } C_i\}$
]

Ý nghĩa của clustering này là: các client đang học cùng loại tấn công hoặc có biểu diễn neuron tương tự sẽ cộng tác mạnh hơn. Ngược lại, client có môi trường quá khác hoặc activation signature lệch sẽ bị giảm trọng số hoặc không được aggregate cùng cụm.

Điểm mạnh so với clustering dựa trên gradient thuần túy là NICE capsule ổn định hơn, vì nhiều activation signature đến từ neuron trưởng thành đã bị đóng băng. NICE cũng chỉ ra rằng activation pattern của neuron có thể phân biệt context quen thuộc và không quen thuộc, nên dùng nó làm tín hiệu clustering là hợp lý.

7. Age-aware decentralized aggregation

Sau khi hình thành cụm, mỗi client aggregate với neighbor trong cụm thay vì với toàn mạng.

Client (C_i) nhận update ($\Delta\theta_j^t$) từ các neighbor ($C_j \in \mathcal{G}_i^t$). Trọng số aggregation được tính:

$$\alpha_{ij} = \frac{s_{ij} \cdot n_j^t \cdot R_j}{\sum_{k \in \mathcal{G}_i^t} s_{ik} \cdot n_k^t \cdot R_k}$$

Nhưng aggregation được mask bởi thông tin tuổi neuron:

$$\theta_i^{t+1} = \theta_i^t + \eta \sum_{j \in \mathcal{G}_i^t} \alpha_{ij} \cdot \mathcal{M}_{ij}^t \cdot \Delta\theta_j^t$$

Trong đó $\mathcal{M}_{ij}^t$ là **NICE-compatible aggregation mask**. Mask này chỉ bật khi các điều kiện sau thỏa mãn:

1. Tham số thuộc vùng plastic hoặc vùng cùng context.
2. Neuron age tương thích giữa hai client.
3. Output label tương ứng cùng semantic class.
4. Activation prototype không lệch khỏi cluster centroid.
5. Update không cố gắng sửa neuron đã trưởng thành của client khác.

Nói ngắn gọn: **không aggregate toàn bộ mô hình**. Chỉ aggregate phần có khả năng chia sẻ tri thức mà không phá vỡ memory cũ.

Với các neuron trưởng thành, có hai lựa chọn an toàn:

- Không aggregate weight của neuron trưởng thành, chỉ aggregate context capsule.
- Chỉ aggregate mature weights nếu hai client có cùng class/task, cùng activation signature và cùng age alignment.

Lựa chọn thứ nhất an toàn hơn cho bài báo, vì nó giữ đúng tinh thần continual learning: tri thức cũ đã ổn định thì không nên bị trung bình hóa tùy tiện.

Để tăng robustness, aggregation trong cụm có thể dùng coordinate-wise median, trimmed mean hoặc geometric median thay vì weighted average. Ngoài ra, update bị xem là đáng ngờ nếu nó làm thay đổi mạnh neuron trưởng thành hoặc có activation signature lệch xa cluster centroid.

8. Context-aware inference

Khi client nhận mẫu mới (x), nó không cần biết task ID. Quy trình suy luận:

1. Tính activation toàn mạng.
2. Chuyển thành binary context vector ($b_i(x)$).
3. Context-detector dự đoán task/context phù hợp:

$$\begin{aligned} &[\\ \hat{e} &= \arg\max_e p_i(e|b_i(x)) \\ &] \end{aligned}$$

4. Mask các output neuron không thuộc context/age tương ứng.
5. Phân loại trong tập class đã học:

[

$\hat{y}$

$\arg\max_{y \in \mathcal{Y}_{\text{seen}}}$
 $f_i(x)_y$
]

Điểm cần nhấn mạnh: hệ thống dự đoán trong không gian các lớp đã được học. Nó không cố gắng phát hiện unknown class. Nếu một lớp tấn công mới được SOC/client gán nhãn và đưa vào task mới, hệ thống sẽ học tiếp lớp đó trong vòng continual learning kế tiếp.

9. Quy trình tổng thể

Có thể mô tả thuật toán như sau:

Input: decentralized clients, local labeled task streams, communication graph, model architecture, threshold (τ), local epoch interval (p).

Tại mỗi client (C_i):

1. Thu thập và gán nhãn dữ liệu task mới (D_i^t).
 2. Mở neuron dự trữ thành neuron trẻ.
 3. Huấn luyện local IDS trên (D_i^t) với freezing/pruning theo NICE.
 4. Định kỳ chọn neuron quan trọng dựa trên activation.
 5. Cập nhật context memory và context-detector.
 6. Khi task kết thúc, tăng tuổi neuron được chọn và đóng băng neuron trưởng thành.
 7. Tạo NICE Context Capsule.
 8. Trao đổi capsule và update với neighbor.
 9. Tính context similarity và hình thành cụm cộng tác phi tập trung.
 10. Thực hiện age-aware robust aggregation trong cụm.
 11. Cập nhật mô hình local và tiếp tục task tiếp theo.
-

10. Điểm mới có thể claim trong bài

Có ba đóng góp chính nên nhấn mạnh.

Thứ nhất, phương pháp đưa NICE vào IDS liên tục để xử lý catastrophic forgetting mà không cần replay dữ liệu cũ. Điều này hợp với IDS vì dữ liệu tấn công cũ có thể nhạy cảm, lớn, hoặc không được phép lưu lâu.

Thứ hai, phương pháp mở rộng NICE sang decentralized FL bằng cách biến activation pattern, age mask và context-detector thành tín hiệu phục vụ clustering. Đây là điểm mới hơn việc chỉ dùng NICE như local continual learner.

Thứ ba, phương pháp đề xuất age-aware aggregation, trong đó neuron trưởng thành được bảo vệ, neuron trẻ được chia sẻ có điều kiện, còn client không tương thích context sẽ không bị ép aggregate chung. Điều này giải quyết một điểm yếu lớn của decentralized FL: aggregation sai cụm có thể làm mô hình IDS suy giảm nghiêm trọng trong môi trường non-IID.

11. Các giả định cần viết rõ để tránh bị phản biện

Cần nói thẳng các giả định sau, vì nếu bỏ qua sẽ dễ bị reviewer bắt lỗi.

Một là, client tự thu thập và gán nhãn dữ liệu task mới. Phương pháp không giải quyết chất lượng gán nhãn, label noise hay unknown-class discovery.

Hai là, các client cần **dùng cùng kiến trúc backbone** hoặc ít nhất cùng không gian neuron/channel đã được định danh. Nếu kiến trúc khác nhau, age-aware parameter aggregation sẽ không trực tiếp áp dụng; khi đó chỉ có thể aggregate ở mức prototype/logit/knowledge distillation.

Ba là, nhãn cần được chuẩn hóa qua một label registry hoặc ontology. Ví dụ “DDoS”, “DoS-Hulk” và “HTTP-Flood” cần được ánh xạ rõ ràng nếu muốn aggregate output head. Đây không phải unknown-class problem, mà là vấn đề đồng bộ ngữ nghĩa nhãn.

Bốn là, **NICE có nguy cơ cạn capacity khi số task quá lớn**. NICE gốc cũng nêu hạn chế này và gợi ý các hướng như tái phân bổ capacity hoặc tích hợp replay giới hạn. Trong bài của mình, nên có cơ chế capacity monitor: nếu layer cạn neuron dự trữ, hệ thống mở rộng layer, tái sử dụng neuron ít quan trọng, hoặc chỉ cho phép học ở tầng cao hơn.

Đề xuất nên nâng cấp thành: Capacity-Aware Neurogenesis Controller

Capacity-Aware Neurogenesis Controller, CANC.

Module này theo dõi capacity của từng layer tại mỗi client và quyết định chiến lược mở rộng hoặc tái phân bổ neuron trước khi học task mới.

Với mỗi client (C_i), tại layer (l), định nghĩa:

$$\left[\rho_{i,l}^0 = \frac{|N_{i,l}^0|}{|N_{i,l}|} \right]$$

là tỷ lệ neuron dự trữ còn lại.

$$\left[\rho_{i,l}^m = \frac{|N_{i,l}^{\geq 2}|}{|N_{i,l}|} \right]$$

là tỷ lệ neuron đã trưởng thành/đóng băng.

$$\left[u_{i,l}^t = \frac{S_{i,l}^t}{|N_{i,l}^1|} \right]$$

là mức tiêu thụ neuron trẻ cho task hiện tại.

Ngoài ra nên thêm một chỉ số quan trọng hơn, gọi là **capacity pressure**:

[

$$\kappa_{i,l}^t$$

$$\begin{aligned} & \alpha(1 - \rho_{i,l}^0) \\ & + \\ & \beta u_{i,l}^t \\ & + \\ & \gamma \Delta \mathcal{L}_{i,l}^{val} \\ & + \\ & \delta d(P_i^t, P_i^{old}) \end{aligned}$$

Trong đó ($\Delta \mathcal{L}_{i,l}^{val}$) đo suy giảm validation khi học task mới, còn ($d(P_i^t, P_i^{old})$) đo độ lệch context prototype của task mới so với các task cũ.

Ý nghĩa: không nên chỉ nhìn “còn bao nhiêu neuron rỗng”, vì có trường hợp neuron dự trữ còn ít nhưng task mới vẫn gần task cũ và học ở tầng cao là đủ. Ngược lại, còn neuron nhưng domain shift mạnh thì vẫn cần mở rộng tầng sớm.

Chính sách quyết định theo thứ tự an toàn

1. Nếu capacity còn đủ: dùng NICE gốc

Nếu:

$$\left[\begin{array}{l} \rho_{i,l}^{(0)} \geq \epsilon_l \end{array} \right]$$

ở hầu hết các layer quan trọng, client tiếp tục học theo NICE: mở neuron ($a=0$), chọn neuron quan trọng theo activation, rồi làm trưởng thành neuron được giữ lại. Đây là chế độ mặc định.

2. Nếu layer thấp cạn nhưng context không đổi mạnh: chỉ học tầng cao

Đây là lựa chọn hợp lý **chỉ khi** task mới vẫn thuộc cùng miền đặc trưng. Ví dụ vẫn là flow-based IDS trong cùng hệ thống mạng, chỉ thêm biến thể tấn công mới. Khi đó layer thấp có thể đã học feature tổng quát như protocol pattern, packet statistics, flow duration, byte rate, connection behavior. NICE gốc cũng quan sát rằng layer sớm thường bị đóng băng nhanh, nhưng điều này không nhất thiết làm giảm hiệu quả nếu các lớp tương lai vẫn cùng domain .

Điều kiện nên viết:

$$\left[\begin{array}{l} d(P_i^t, P_i^{\text{old}}) < \xi \\ \quad \text{\text{and}} \quad \\ \rho_{i,l_{\text{low}}}^{(0)} < \epsilon_l \end{array} \right]$$

Khi đó chỉ cho phép học ở các layer cao hoặc classifier head:

$$\left[\begin{array}{l} \theta_{i,l_{\text{low}}} \rightarrow \text{frozen}, \quad \\ \theta_{i,l_{\text{high}}} \rightarrow \text{plastic} \end{array} \right]$$

Điểm cần tránh: không nên viết “nếu cạn layer thì chỉ học tầng cao” như một luật tổng quát. Nếu task mới là domain shift mạnh, ví dụ từ enterprise flow sang IoT/ICS traffic, cách này có thể thất bại.

3. Nếu domain shift mạnh: mở rộng có kiểm soát

Nếu context prototype lệch mạnh:

$$\left[\begin{array}{l} d(P_i^t, P_i^{\text{old}}) \geq \xi \end{array} \right]$$

và layer sớm thiếu capacity, khi đó client cần mở rộng layer. Nhưng trong decentralized FL, mở rộng tùy tiện sẽ làm các client không còn cùng kiến trúc, gây khó aggregate. Vì vậy nên dùng **sparse expansion block** hoặc **adapter expansion** thay vì mở rộng toàn bộ layer.

Ví dụ thêm một nhánh nhỏ:

$$\begin{bmatrix} z_l = f_l(z_{l-1}) + A_{\{i,l\}}^{\text{new}}(z_{l-1}) \end{bmatrix}$$

Trong đó $(A_{\{i,l\}}^{\text{new}})$ là adapter/task-specific expansion block. Khi aggregate, chỉ aggregate adapter giữa các client có cùng architecture version và context cluster tương thích.

Cần thêm metadata:

$$\begin{bmatrix} v_i^t = \text{architecture version} \end{bmatrix}$$

vào NICE Context Capsule. Client chỉ aggregate parameter nếu:

$$\begin{bmatrix} v_i^t = v_j^t \end{bmatrix}$$

hoặc nếu có mapping rõ ràng giữa các adapter tương ứng.

4. Tái sử dụng neuron ít quan trọng: chỉ dùng như graceful forgetting

Đây là phần nguy hiểm nhất. Nếu reset neuron trưởng thành rồi dùng lại, claim “mature neurons preserve old knowledge” sẽ bị yếu. NICE gốc bảo vệ neuron già bằng freezing/pruning để tránh thay đổi tri thức cũ. Vì vậy không nên gọi là “tái sử dụng neuron ít quan trọng” một cách trực tiếp, mà nên gọi là:

controlled neuron retirement hoặc **graceful capacity recycling**.

Neuron chỉ được tái sử dụng nếu thỏa mãn đồng thời:

$$\begin{bmatrix} l_{\{i,l,u\}}^{\text{old}} < \eta_1 \end{bmatrix}$$

$$\begin{bmatrix} \text{usage}_{\{i,l,u\}}^{\text{recent}} < \eta_2 \end{bmatrix}$$

[
 $\Delta F1_{old} < \eta_3$
]

Tức là neuron đó ít kích hoạt cho task cũ, ít được context-detector dùng gần đây, và việc loại bỏ nó không làm giảm đáng kể hiệu quả trên validation/prototype memory cũ.

Nên có trạng thái trung gian:

[
 $a = -1$
]

gọi là **retired neuron**. Neuron này chưa bị dùng lại ngay, mà được đưa vào vùng kiểm tra. Nếu sau một số round không gây suy giảm trên các prototype/context memory cũ, nó mới trở lại ($a=0$).